Big Claims Require Big Evidence

Document Number:	DxxxxR0
Date:	2026-03-14
Audience:	WG21 Plenary, Direction Group
Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Large-scope proposals carry large-scope risk. When a framework claims universality - applicability across multiple domains - the committee's review process must scale with the claim. This paper identifies a recurring pattern in which a framework excels in its core domain, claims an adjacent domain without proportional evidence, and the adjacent domain ships nothing while experts leave and users wait. A case study drawn from the networking gap illustrates the pattern. Six structural failure modes are diagnosed. Five safeguards are proposed. A checklist of thirteen questions is offered for application to any large-scope proposal at any stage. A shared vocabulary of thirteen named concepts is introduced to give the committee a common language for dynamics it already experiences.

This paper names no individuals. It proposes no wording. It proposes process.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

The Pattern

A framework is proposed. It is good engineering. It solves real problems in a real domain. The committee reviews it, finds it sound, and invests years of design work.

The framework then claims an adjacent domain. The claim is plausible. The framework's abstractions are general. A small example demonstrates that the adjacent domain's operations can be expressed in the framework's vocabulary. The committee accepts the claim and proceeds.

Years pass. Experts in the adjacent domain publish papers identifying structural difficulties. The framework's revision history acknowledges the difficulties. The difficulties are not resolved. New revisions ship with the acknowledgment intact and the problem open.

A direction poll is taken. Two options are presented. The framework is one. The incumbent design for the adjacent domain is the other. A third option - one that has implementation experience but no political constituency - is not among the choices. The framework wins. The incumbent is removed.

The adjacent domain now depends on the framework. The structural difficulties remain. The framework's authors publish examples. None are in the adjacent domain. The committee's investment creates momentum. Voting against the framework feels like wasting years of work. The feature advances.

The expert who built the most widely deployed library in the adjacent domain stops attending. Users in the adjacent domain wait. Competing languages ship the facility. The committee ships nothing.

The pattern is not specific to any one feature. The reader may recognize it.

The Case Study

The following timeline uses only paper numbers, dates, and paraphrased observations. No individuals are named.

- **2021.** A paper (P2430R0) identifies that the framework's channel model cannot represent partial-success results - operations that return both a status code and associated data - without losing data, bypassing the channels, or converting routine outcomes into exceptions. The paper's author is the designer of the most widely deployed networking library in C++.
- **2021.** A second paper (P2471R1) observes that translating completion tokens to the framework's model requires heuristics to determine which argument is an error and which is data. The heuristic is ambiguous for compound results.
- **2021.** The framework's authors publish a new revision that adds a section on partial success. The section acknowledges the problem. It does not resolve it.
- **2021.** On the committee reflector, a member observes that abstracting I/O into a concurrency model leads to inferior outcomes compared to abstracting I/O as the kernel implements it - a submission-completion queue - and then bridging to the concurrency model. The member notes a lack of resources to write the paper.
- **2021.** On the committee reflector, the framework's lead author posts a small third-party library as an existence proof that the framework can express networking APIs. The library is not production-scale. A senior committee member responds that the question is not whether the framework can do networking but how well the result fits the model and what benefits the model provides.
- **2021.** The framework's lead author states on the reflector that the vast majority of uses will be in coroutines, not in the framework's algorithmic sub-language.
- **2023.** A paper (P2762R2) documents five channel-routing options for I/O and acknowledges that the single-argument error channel is "somewhat limiting" for partial-success scenarios. The paper does not resolve the difficulty.
- **2023.** At a plenary meeting, a direction poll presents two choices for networking: the incumbent design or the framework. A coroutine-native approach - which has implementation experience - is not among the choices. The framework wins. The incumbent design is removed from consideration.
- **2024.** The framework's lead author publishes seven examples demonstrating the framework's use. The examples cover thread pools, embedded systems, cooperative multitasking, and custom algorithms. None involve networking. None involve I/O.

- **2025.** A paper (P3801R0) documents that the framework's coroutine task type lacks symmetric transfer support - the mechanism that prevents stack overflow in deep coroutine chains. The author describes a thorough fix as "non-trivial."
- **2026.** A domain study group polls the statement that compound I/O results present a design challenge for the framework's channel model. The poll fails to reach consensus.
- **2026.** Within twenty-four hours of the poll, multiple members confirm on the committee reflector that no standard facility exists for dispatching compound results onto the framework's channels based on a runtime choice. The framework's own task type author describes his dispatch mechanism and expresses dissatisfaction with it. A member posts four working implementations and confirms that all four are equivalent to a single line of coroutine code. Another member proposes a sender adapter to solve the problem - an adapter that does not exist and has not been designed.
- **2026.** The author of the most widely deployed C++ networking library no longer attends WG21 meetings. C++ users do not have standard networking. Python, Go, Rust, Java, and C# do.

The partial-success problem was identified in 2021. It was acknowledged in 2021. It was documented again in 2023. It was confirmed unresolved in 2026. Five years. The same problem. Never fixed.

The Structural Diagnosis

Six failure modes enabled the pattern.

No burden of evidence for scope claims. The framework claimed applicability to networking. The claim was accepted on the basis of a small third-party library and the general expressiveness of the framework's abstractions. No production-scale networking implementation using the framework's channel model was required. No constructed comparison between the framework and the incumbent was demanded. The claim advanced on plausibility, not evidence.

Acknowledged-but-unresolved problems have no expiration. The framework's revision history acknowledged the partial-success problem in 2021. Five years and multiple revisions later, the problem remained open. No committee mechanism required resolution before the feature advanced. Acknowledgment substituted for action.

Binary polls exclude emergent alternatives. The 2023 direction poll presented two options. A third approach - coroutine-native I/O - had implementation experience and published papers but was not among the choices. The committee voted between two known positions and excluded a third that had evidence but no political constituency.

Process momentum replaces technical review. After years of design review, voting against the framework felt like wasting the committee's investment. The feature had momentum. Momentum is not merit. But in a consensus process, they are difficult to distinguish.

Employer concentration distorts room composition. When a single employer's engineers constitute a significant fraction of a study group's attendance, the employer's priorities can dominate direction polls and design reviews. This is not malicious. Engineers naturally advocate for the designs their employer uses and funds. But the effect is that room composition reflects employer interest rather than domain expertise.

Too big to review. The framework specification spans hundreds of pages. The domain expertise required to evaluate its claims in networking, GPU dispatch, thread pool management, and coroutine integration exceeds what any single reviewer possesses. When a proposal is too large for independent verification, the committee substitutes trust in the authors' standing for technical review. Trust is a social relationship. It is influenced by institutional standing, employer backing, conference presence, and reputation. None of these correlate with correctness.

The Commons

The C++ standard is shared infrastructure. Every employer that sends engineers to WG21 depends on the standard being healthy. The standard is a commons.

Engineers naturally advocate for designs their employer uses and funds. This is not malicious. It is rational. But when one employer's priorities capture a design direction, the commons degrades. The framework may serve the capturing employer's domain. It may not serve the adjacent domain. The adjacent domain ships nothing. Users in that domain wait, or leave, or choose another language.

The employer that captured the design direction has won a battle and damaged the field on which all future battles are fought.

The short-term gain - a framework shaped to one employer's use case, advanced through the committee by that employer's engineers - comes at the cost of long-term ecosystem health. The capturing employer depends on that ecosystem. Its customers write networking code. Its products serve data over networks. Its inference workloads depend on I/O. When C++ networking remains non-standard because the framework that claimed networking could not deliver it, the capturing employer's own customers suffer.

Every employer that depends on C++ depends on the commons being healthy. The safeguards proposed in the next section are not punitive. They protect the commons that every employer needs.

The Safeguards

Five process changes would have surfaced the case study's problems years before they became irreversible.

Scope substantiation. Any paper that claims applicability to multiple domains must demonstrate the claim in each domain with production-scale evidence before advancing past design review. A constructed comparison - the best possible implementation in both the framework and the alternative, evaluated on the hardest problems in the claimed domain - satisfies the requirement. A toy library does not. Slide evidence is not evidence.

Resolution countdown. When a design review identifies a structural problem and the problem is acknowledged in the paper's revision history, a clock starts. If the problem is not resolved within two revision cycles, the feature cannot advance past its current stage until the problem is addressed. The clock is public and tracked. Acknowledgment is not resolution.

Alternative inclusion. When a direction poll is taken, any alternative approach with published implementation experience must be included as an option. The chair cannot present a binary choice when a third option exists in the mailing with a working implementation. The binary poll is a structural exclusion mechanism. It must be closed.

Domain veto. When a feature claims applicability to a domain, the relevant domain study group must poll on the claim before the feature advances. A failed poll in the domain study group does not block the feature in its core domain. It removes the scope claim for the contested domain. The feature ships for what it can demonstrate. It does not ship for what it merely claims.

Adoption ladder. For proposals above a complexity threshold, evidence must demonstrate layered adoption: the authors built it, independent developers used it, independent libraries were built on top of it, independent applications were built on those libraries, and real users - not employees of the proposing organization - shipped production code. Large proposals should also be decomposed into independently reviewable units so that review is verification, not trust.

The Checklist

The following questions, applied to any large-scope proposal at any stage, would have surfaced the case study's problems years before they became irreversible. Every question answers "yes" for the case study.

Scope

- Does this paper claim applicability beyond its demonstrated domain?
- Has the paper demonstrated production-scale use in every claimed domain?
- Is there a domain expert for each claimed domain who has reviewed and endorsed the claim?

Evidence

- Is the implementation experience from independent developers, or only from the authors and their employer?
- Has anyone built a library on top of this? Has anyone built an application on top of that library?
- Are the examples in the paper representative of the hardest problems in the claimed domain, or only the easiest?

Unresolved problems

- Does the revision history acknowledge a problem that remains unresolved?
- How many revision cycles has the unresolved problem survived?
- Has a domain study group polled on whether the problem is real?

Alternatives

- Are there alternative approaches with implementation experience that were not polled?
- Was the direction poll binary when a third option existed?

Human cost

- Has a domain expert stopped participating because of a direction decision?
- Are users in the claimed domain still waiting for a standard facility that competing languages already provide?

The Language of WG21

The following terms name dynamics the committee already experiences but has no shared vocabulary for.

Term	Definition
Big Claims, Big Evidence	Scope claims must be substantiated proportionally to their breadth
Review by Reputation	When a proposal is too large for independent verification and the committee substitutes trust in the authors for technical review
The Binary Poll	When a direction vote presents two options and excludes a third that has implementation experience but no political constituency

Term	Definition
Process Momentum	The force that makes voting against a feature feel like wasting the committee's investment, even when the feature is not ready
Scope Creepage	A framework claims one domain, ships, then claims the next using "we are already in the standard" as leverage; the claim only expands, never contracts
Slide Evidence	When a toy implementation is cited as evidence that a framework works in a domain, substituting "it compiles" for "it works at scale"
The Adoption Ladder	Authors built it, independents used it, libraries built on it, applications built on those, real users shipped production code
The Empty Seat	When a domain expert stops attending because the committee chose a path that discarded their work; the absence is evidence the process failed
Captured Commons	When one employer's priorities dominate a design direction, winning a battle and damaging the field on which all future battles are fought
Resolution Countdown	A known problem starts a timer; if not resolved within two revision cycles, the feature cannot advance
Domain Veto	When a domain study group polls against a framework's applicability to their domain, the scope claim is removed for that domain
The Constructed Comparison	Build the best possible version in both models and let the comparison table speak
Scope the Claim, Ship the Core	A framework should scope its claim to the domains where it has evidence, then ship for those domains and explicitly disclaim the rest

Closing

If the committee had required the framework to demonstrate production-scale networking with its channel model before claiming networking as a domain - if big claims had required big evidence - would we have standard networking today?

The author believes we would. The framework would have shipped for its core domain: GPU dispatch, compile-time work graphs, thread pool submission, timer management. The networking library would have shipped alongside it, using the design that twenty years of deployment had validated. The two would have coexisted,

with explicit bridges at the boundaries. The expert who built that networking library would still be in the room.

The committee cannot undo the past. It can change the process so that the pattern does not repeat. Contracts, reflection, and every large-scope feature that follows will face the same dynamics: scope claims, process momentum, employer concentration, and proposals too large for independent review. The safeguards, the checklist, and the shared vocabulary in this paper are offered as tools for recognizing the pattern before the cost is paid.

Big claims require big evidence. The evidence was not required. The cost was paid by others.